IT & BUSINESS COLLEGE

Department: Computer Science

Project title: Hermes

Subject: Introduction to Engineering and Informatics

Members:

Asylbekov Islambek – ID238715004

Anarbekov Azimbek – ID238711048

Turganaliev Diyaz – ID238715003

Bekmuhambetov Daniyar – ID238715001

Instructor: Mirlan Nurbekov

Group: SCA-23A

Contents

Table of Figures

Abstract

The rapid growth of mobile technologies has significantly transformed modern transportation services. In recent years, shared mobility platforms have become increasingly popular as an alternative to traditional vehicle ownership. Car-sharing systems allow users to access vehicles temporarily without the need to purchase or maintain their own cars.

This project presents the development of Hermes, a mobile car-sharing application prototype designed to simplify the vehicle rental process through a mobile platform. The application allows users to register, verify their identity, browse available cars, select pickup locations, and reserve vehicles directly from their smartphones.

The Hermes system integrates several technologies, including flutter for mobile development, java for backend services, and PostgreSQL for database management. Communication between the mobile application and the backend server is implemented using RESTful API endpoints.

The purpose of the Hermes prototype is to demonstrate how modern software technologies can be combined to create a functional car-sharing system. The project illustrates the full development process, including system analysis, design, implementation, testing, and documentation.

1. Introduction

1.1 Background

Urban transportation has undergone significant changes in recent years due to the rapid development of digital technologies. Many cities face challenges related to traffic congestion, high vehicle ownership costs, and limited parking space. As a result, alternative transportation solutions have become increasingly important.

One of the most promising solutions is the concept of car-sharing systems. Car-sharing platforms allow users to temporarily rent vehicles when needed instead of owning a car. This approach reduces transportation costs for users and helps decrease the number of privately owned vehicles on the road.

Modern car-sharing services rely heavily on mobile applications that allow users to quickly locate and reserve vehicles. These applications typically integrate several technological components including mobile interfaces, backend servers, databases, and location services.

1.2 Purpose of the Project

The purpose of the Hermes project is to develop a prototype of a mobile carsharing system that demonstrates the basic functionality of modern car rental platforms.

The main goals of the project include:

- designing a user-friendly mobile application
- implementing a secure authentication system
- creating a vehicle inventory system
- implementing a booking and rental process
- managing vehicle availability
- demonstrating communication between frontend and backend systems

The Hermes prototype provides a simplified version of a real-world carsharing service while demonstrating key software engineering concepts.

2. Problem Statement and Objectives

2.1 Problem Statement

In many urban areas, access to transportation is often inefficient due to the high costs associated with vehicle ownership. Purchasing a car requires significant financial investment, including maintenance, fuel, insurance, and parking expenses.

Traditional car rental services also present several limitations. Many rental companies require users to complete lengthy registration procedures, provide physical documents, and visit physical offices to obtain vehicles. These processes can be inconvenient for users who require quick access to transportation.

Additionally, the lack of flexible rental options can make it difficult for users to rent vehicles for short periods of time.

Car-sharing systems address these challenges by providing digital platforms that allow users to locate and rent vehicles through mobile applications. However, developing such systems requires the integration of multiple technological components including authentication systems, booking logic, and database management.

The Hermes project aims to demonstrate how such a system can be implemented using modern software development tools.

2.2 Project Objectives

The main objectives of the Hermes project are listed below.

Primary Objectives

- develop a comprehensive mobile application tailored for vehicle rental services.
- implement a secure authentication system to protect user credentials and restrict unauthorized access.
- design a dynamic vehicle inventory system to efficiently manage fleet availability.
- engineer a seamless vehicle booking process for end-users.
- integrate location-based pickup points to facilitate convenient vehicle handovers.

Secondary Objectives

- demonstrate robust integration between the mobile frontend interface and the backend architecture.
- deliver an intuitive, user-friendly interface to enhance the overall user experience.
- implement functional rental tracking to monitor booking durations and vehicle status.
- establish reliable data management utilizing a relational database system

3. System Requirements and Constraints

3.1 Functional Requirements

Functional requirements describe the actions that the system must perform.
The Hermes application must allow users to perform the following operations:

User Management

- register a new account
- log into the system
- update personal profile information
- upload driver license information

Vehicle Management

- view a list of available vehicles
- view detailed vehicle information
- search for vehicles

Booking System

- select a vehicle
- choose a pickup location
- reserve a vehicle
- track rental duration

3.2 Non-Functional Requirements

Non-functional requirements define the quality and performance characteristics of the system.

Important non-functional requirements include:

Performance

- the application should respond quickly to user requests
- API requests should return results efficiently

Security

- user authentication must be secure
- sensitive information must be protected

Usability

- the user interface must be simple and intuitive
- navigation between screens should be clear and logical

Reliability

- the system should operate without frequent crashes or errors

3.3 Hardware Requirements

The Hermes application requires the following hardware components:

For development:

- personal computer
- android development environment
- internet connection **For users:**
- smartphone with Android operating system
- internet connectivity

3.4 Software Requirements

To construct a scalable and efficient car-sharing system, the Hermes project utilizes a carefully selected stack of modern software technologies. The mobile client frontend is built using the **flutter framework** and the **dart programming language**, providing a highly responsive and unified user experience across devices.

For the backend infrastructure, the system relies on the **java programming language**, specifically leveraging the Spring Boot framework to construct robust **RESTful API** endpoints. This backend manages core business logic and communicates directly with a **PostgreSQL** relational database, which was chosen to ensure secure and structured storage of user and vehicle data.

The development lifecycle was supported by industry-standard tools, including **figma** for interface design and prototyping, **android Studio** for mobile environment emulation, and **git (via github and gitlab)** for comprehensive version control and code repository management.

4. System Design

4.1 System Architecture

The Hermes system follows a **client-server architecture**.
The system is divided into three main components:
mobile Application
backend Server
database System
The mobile application acts as the client interface used by end users. It sends requests to the backend server using HTTP requests. The backend server processes these requests and retrieves data from the PostgreSQL database.

4.2 Database Design

The ER diagram in **Figure 1** visualizes the relational structure of the PostgreSQL database. It illustrates the primary entities - users, cars, location, and bookings - and the logical associations between them. For instance, the 'one-to-many' relationship between users and bookings ensures that a single client can manage multiple rental history records, while foreign key constraints maintain data integrity across the system

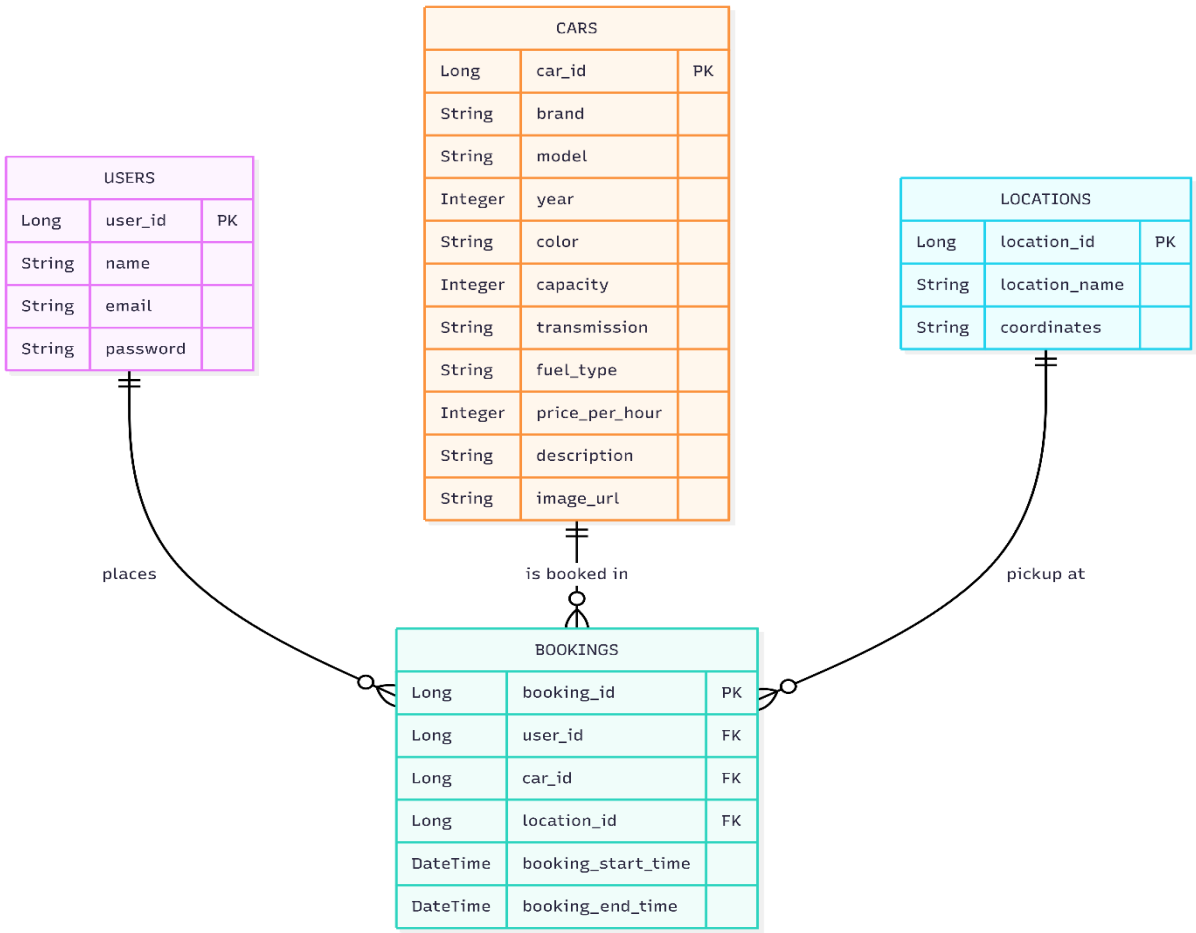


Figure 1. Entity-Relationship Diagram of the Hermes Database.

The main database entities include:

Users

- user_id
- name
- email
- password

Cars

- car_id
- car_name
- year
- color
- capacity
- transmission
- fuel_type
- price_per_hour
- description

Bookings

- booking_id
- user_id
- car_id
- booking_start_time
- booking_end_time

Locations

- location_id
- location_name
- coordinates

4.3 Mobile Application Design

The frontend interface of the Hermes application is designed to deliver a seamless and intuitive user experience. To ensure logical navigation, the application architecture is divided into three primary functional modules:

- **Authentication & Onboarding Module:** Includes secure login and registration interfaces designed to smoothly introduce users to the platform while protecting their credentials.
- **Core Application Flow:** Encompasses the main home dashboard, an interactive vehicle list (catalog), comprehensive vehicle details views, and a streamlined booking interface to facilitate a frictionless rental process.
- **Account Management Module:** Features a profile information dashboard and a secure driver license verification system to handle user data and permissions.

The overall design philosophy strictly prioritizes usability, accessibility, and cognitive simplicity, ensuring that users can easily navigate from initial authentication to final vehicle reservation.

4.4 API Design & Endpoint Specification

The flutter mobile application communicates with the Spring Boot backend using a RESTful API. To keep the system secure and protect user data, the Hermes project uses role-based access control (RBAC). This means that important tasks, like managing vehicles, are only allowed for authorized users (OWNERS). On the other hand, regular features like browsing the catalog are available to all CLIENTS. The full technical details of how this works can be seen in **Figure 2,** which shows the main service paths and security rules.

Method	Endpoint	Required Role	Description
GET	/api/cars	CLIENT / OWNER	Fetches the full catalog of available vehicles for the home screen.
POST	/api/cars/add-car	OWNER / CLIENT	Submits a new vehicle listing including JSON metadata and Multipart image.
GET	/api/cars/{id}	CLIENT / OWNER	Retrieves comprehensive details for a specific car (used in CarDetailsScreen).
POST	/api/bookings	CLIENT	Initiates a rental request for a selected time slot and vehicle.

Figure 2. REST API Specification and Role-Based Access Endpoints.

5. Implementation

The implementation stage represents the practical phase of the Hermes project where the system design is transformed into a working application. During this stage, the development team implemented both the backend infrastructure and the mobile frontend application. The goal of this stage was to create a functional prototype that demonstrates the core functionality of a car-sharing system.

The implementation process required careful coordination between frontend and backend components. The mobile application communicates with the backend server through REST API endpoints, allowing the system to exchange data related to users, vehicles, and bookings.

5.1 Backend Implementation

The backend of the system was implemented using **java**. You can see the structure of backend in **Figure 3**

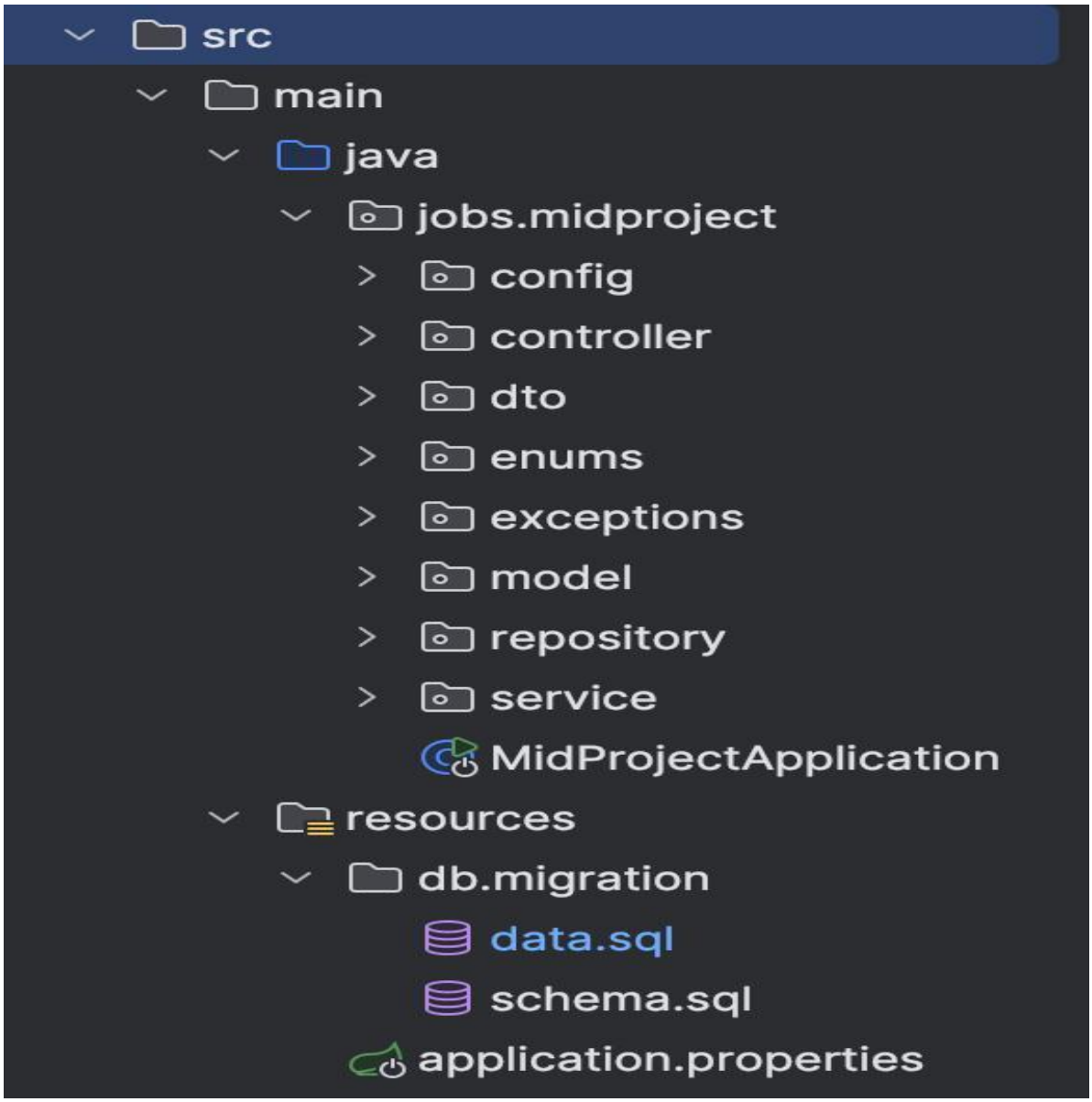


Figure 3. Modular project structure of the Spring Boot backend

The backend is responsible for processing requests from the mobile application, managing the PostgreSQL database, and handling security. A core part of this implementation is the authentication system, which uses JSON web tokens (JWT). After a user logs in with valid credentials, the backend generates a token that the mobile device stores to authenticate all future requests.

Another critical component is the vehicle inventory management system. The server maintains a list of cars available for rent and ensures that all submitted data is accurate. To achieve this, the system uses data transfer objects (DTOs) with specific validation rules. **Figure 4** illustrates how the backend validates vehicle information-such as ensuring the price is positive-before any data is saved to the database.

Additionally, since vehicle registration requires uploading photos, the system includes a controller to handle complex data. **Figure 5** demonstrates how the backend processes these image files together with text details. This combined approach prevents errors and ensures that every car in the catalog has high-quality data and all necessary visual information

```java
@Data  12 usages  Diyaz007 *
@Getter
@Setter
public class CarRequest {

        @NotBlank(message = "Brand is required")
        private String brand;

        @NotBlank(message = "Model is required")
        private String model;

        @NotNull(message = "Year is required")
        private Integer year;

        @NotBlank(message = "Color is required")
        private String color;

        @Min(value = 1, message = "Capacity must be at least 1")
        private Integer capacity;

        @NotBlank(message = "Fuel type is required")
        private String fuelType;

        @NotBlank(message = "Transmission type is required")
        private String transmission;

        @NotBlank(message = "Description is required")
        private String description;

        @DecimalMin(value = "0.0", inclusive = false, message = "Price must be greater than 0")
        private Integer pricePerHour;
}
```

Figure 4. Implementation of CarRequest DTO with Jakarta Validation in Java Spring Boot.

```java
@Tag(name = "Автомобили", description = "API для работы с автомобилями")  Diyaz007 *
@RestController
@RequestMapping("/api/cars")
@RequiredArgsConstructor
public class CarController {
    private final CarService carService;
    private final UserServiceImpl userService;

    @Operation(summary = "Добавить новый автомобиль (с изображением)", description = "Доступно только владельцам (OWNER)")
    @PostMapping(value = "/add-car", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    @PreAuthorize("hasAnyRole('OWNER','CLIENT')")
    public ResponseEntity<?> addCar(
            @ModelAttribute CarRequest carRequest,
            @RequestPart("image") MultipartFile image,
            Authentication authentication){
        User user = userService.getUserByEmail(authentication.getName());
        if(!user.isPersonalInfo()){
            return ResponseEntity.status(HttpStatus.FORBIDDEN).body("Заполните персональные данные");
        }

        CarResponse response = carService.addCar(carRequest,image,user.getEmail());
        return ResponseEntity.ok(response);
    }
```

Figure 5. REST Controller handling Multipart/form-data for vehicle registration.

The backend also implements the **booking logic**. When a user selects a car and initiates the booking process, the backend verifies that the vehicle is available. If the car is available, the system creates a booking record in the database and updates the vehicle's status to prevent other users from renting the same vehicle simultaneously. Additionally, the backend manages **rental duration tracking**. The system records the start time of the booking and calculates the total rental duration when the vehicle is returned.

5.2 Frontend Implementation

The frontend of the Hermes application was developed using the Flutter framework. This choice allowed us to build a high-performance application for both Android and iOS using a single codebase. Flutter was selected because it provides a modern UI toolkit and a fast development cycle, which helped us create a simple and intuitive user interface for our customers. Several key screens were implemented to ensure a smooth user journey. This includes authentication screens for registration and a main dashboard that displays the user's profile and current balance.

For the car browsing experience, we focused on performance and usability. **Figure 6** demonstrates how we used flutter's efficient list-building tools to display the vehicle catalog. By using this approach, the app remains fast and responsive even when the list of available cars grows larger. Beyond the visual elements, a critical part of the development was ensuring the mobile app could correctly process data from the server. To maintain consistency, we implemented strict synchronization between our backend and frontend. **Figure 7** illustrates how data structures from the java backend are mapped into dart models. This synchronization is vital because it ensures that information like vehicle prices and IDs is handled accurately by the mobile application without errors

Figure 6. The main vehicle catalog interface implemented with ListView.builder.

Field (JSON Key)	Java Type (DTO)	Dart Type (Model)	Description
id	Long / String	String	Unique identifier for the vehicle.
brand	String	String	Manufacturer name (e.g., Tesla, BMW).
pricePerHour	Integer	int	Hourly rental rate (Strictly Integer for API consistency).
capacity	Integer	int	Vehicle passenger/load capacity (Min value: 1).
description	String	String	Detailed owner's note about the car's condition.
imageUrl	String	String	Relative path transformed into an absolute URL by the Dart model.

Figure 7. Data Mapping and Type Synchronization between Java DTO and Dart Models.

Profile Screen
Users can update personal information and upload driver license data required for verification.

AddCar Screen
Users can fill the information and upload the picture of their own car to add it to our list and let other users rent it. **Figure 8** shows the interface of our screen where you can add your cars.

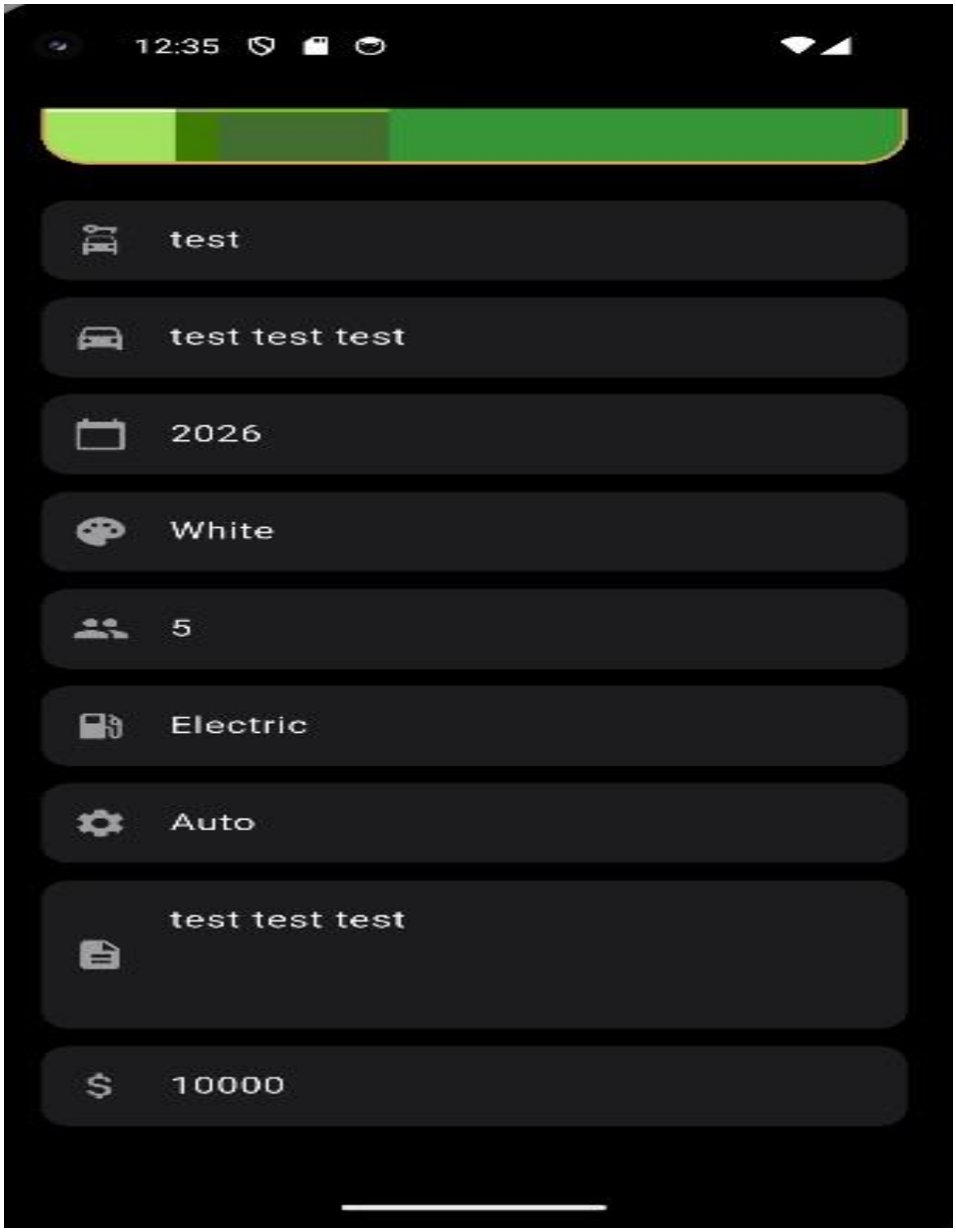


Figure 8. Interactive car registration form with input validation and ImagePicker integration

The frontend communicates with the backend server using HTTP requests. The mobile application sends requests to the API endpoints and processes the JSON responses received from the server. This process is illustrated in **Figure 9**, which shows how the app converts raw data from the API into dart objects. Specifically, it demonstrates the logic for mapping image URLs, ensuring that car photos are correctly loaded and displayed in the user interface**.**

```
factory Car.fromJson(Map<String, dynamic> json) {
  const String backendBaseUrl = 'https://ungrudging-carson-nonvituperatively.ngrok-free.dev';
  final rawImage = (json['imageUrl'] ?? json['image'] ?? '').toString();
  final imageUrl = rawImage.isEmpty
      ? ''
      : (rawImage.startsWith('http')
      ? rawImage
      : (rawImage.startsWith('/')
      ? '$backendBaseUrl$rawImage'
      : '$backendBaseUrl/$rawImage'));

  final rawId = json['id'] ??
      json['_id'] ??
      json['carId'] ??
      json['uuid'] ??
      json['ID'];
  final id = rawId == null ? '' : rawId.toString();

  return Car(
    id: id,
    bookingCarId: _parseBookingCarId(json),

    brand: (json['brand'] ?? '').toString(),
    model: (json['model'] ?? json['title'] ?? '').toString(),

    description: (json['description'] ?? 'No description provided.').toString(),
    capacity: (json['capacity'] as num?)?.toInt() ?? 1,

    rating: (json['rating'] as num?)?.toDouble() ?? 4.7,
    imageUrl: imageUrl,
```

Figure 9. JSON Deserialization logic and image URL mapping in the Flutter Car model.

5.3 Database Implementation

The Hermes system uses **PostgreSQL** as its database management system. PostgreSQL was selected because it is a powerful open-source relational database that provides strong data integrity and performance.

The database stores all important information related to the application.

Several tables were created to organize system data.

Main database tables include:

Users Table

Stores user account information including name, email, password, and driver license details.

Cars Table

Stores vehicle information such as car model, rental price, vehicle type, and availability status.

Bookings Table

Stores information about rental transactions including which user rented which vehicle and the duration of the rental.

Locations Table

Stores predefined pickup locations that users can select when booking a vehicle.

The database interacts with the backend server through SQL queries that allow data to be inserted, retrieved and updated as necessary.

6. Testing and Results

Testing is an essential stage of software development that ensures the reliability and functionality of the system. During the testing phase of the Hermes project, the development team evaluated different parts of the system to ensure that all components work together correctly.

As demonstrated in **Figure 10**, the systematic evaluation of the Hermes platform yielded successful results across all critical modules. The integration between the flutter frontend and the spring boot backend proved to be stable under various test scenarios. Crucially, the system successfully handled edge cases, such as preventing duplicate bookings and blocking unauthorized access attempts. These results confirm that the developed prototype fully satisfies the functional requirements and security constraints outlined in the initial system design.

Test Case ID	Module	Action (Input / Steps)	Expected Result	Actual Result	Status
TC-001	Authentication	User attempts to log in with a valid registered email and password.	System validates credentials, generates a JWT token, and grants access.	JWT successfully generated; user navigated to Home Screen.	Pass
TC-002	Authentication	User attempts to log in with an incorrect password.	System denies access, returns 401 Unauthorized, and displays an error message.	Access denied; error message correctly displayed on UI.	Pass
TC-003	Security (RBAC)	A user with the CLIENT role attempts to send a POST request to /api/cars/add-car.	Spring Security intercepts the request and returns 403 Forbidden.	Request blocked by backend; 403 status verified via Postman.	Pass
TC-004	Data Validation	Owner tries to register a car with a negative pricePerHour value.	Jakarta Validation triggers, rejecting the payload with a 400 Bad Request.	Form submission blocked; backend returns detailed validation error.	Pass
TC-005	Vehicle Mgmt	Owner submits a valid CarRequest DTO along with a vehicle photo (multipart/form-data).	Vehicle data is saved to PostgreSQL, image is processed, and system returns 200 OK.	Vehicle successfully appears in the mobile catalog with the correct image.	Pass
TC-006	Booking Logic	Client selects an available vehicle and confirms the reservation.	Booking record is created in the database, and the vehicle's availability status is updated.	Booking confirmed; vehicle is immediately removed from the 'Available' list.	Pass
TC-007	Booking Logic	Two clients attempt to book the exact same vehicle simultaneously.	System prevents double-booking, confirming the first request and rejecting the second.	Database constraint prevents duplicate booking; second user receives an error.	Pass
TC-008	API Integration	Mobile application requests the vehicle catalog via the secure Ngrok tunnel.	Backend processes the GET request and returns a JSON list of available cars.	Flutter UI successfully parses JSON and populates the ListView.builder.	Pass

Figure 10. System Testing Matrix and Core Functional Test Cases.

6.1 Authentication Testing

The security and integrity of user data are paramount in the Hermes car-sharing platform. Therefore, comprehensive authentication testing was conducted to validate the robustness of the registration and login mechanisms. This testing phase aimed to ensure that the backend securely manages user identities and effectively guards against unauthorized access.
The following critical security scenarios were systematically tested:

- **User registration and onboarding:** Verifying the successful creation of new user accounts. This included testing the secure transmission of user credentials and ensuring that new profiles are correctly inserted into the PostgreSQL database without data loss.

- **Credential verification and error handling:** Simulating authentication requests using both valid and invalid credentials. This test confirmed that the system accurately grants access to legitimate users while correctly rejecting incorrect passwords or unregistered emails, returning appropriate HTTP status codes (e.g 401 unauthorized).

- **JWT generation and lifecycle management:** Confirming that upon successful authentication, the Spring Boot backend correctly generates a secure JSON web token (JWT). Testing also verified that the flutter mobile frontend securely receives, parses, and stores this token locally for continuous session management.

- **Endpoint security and access control:** Testing the system's ability to protect sensitive REST API endpoints. This involved attempting to access restricted resources (such as vehicle management functions) without a valid token or with an insufficient user role, ensuring the system strictly enforces role-based access control (RBAC). The outcomes of these test scenarios confirmed the high reliability of the platform's authentication architecture. The system successfully demonstrated its capability to maintain secure user sessions, properly handle token routing, and strictly prohibit unauthorized entities from accessing restricted parts of the system.

6.2 API Communication Testing

Since the Hermes application uses a client-server architecture, proper communication between the mobile application and backend server is critical.

API testing included verifying that the mobile application can successfully send requests to the backend server and receive correct responses.

The following operations were tested:

- retrieving the list of available vehicles
- sending booking requests
- retrieving user profile information
- updating user profile data

The results showed that the REST API endpoints function correctly and return appropriate data to the mobile application.

To facilitate communication between the physical android device and the local spring boot server, a secure ngrok tunnel was configured as you can see in **Figure 11**. Since the backend services were hosted on a local environment (localhost:8080), they were not natively accessible to external devices. By implementing a secure HTTPS tunnel, the local API was exposed via a public URL, allowing the Flutter application to perform real-time data exchange through the internet. This configuration was essential for testing the end-to-end functionality of the car-sharing system under realistic network conditions.

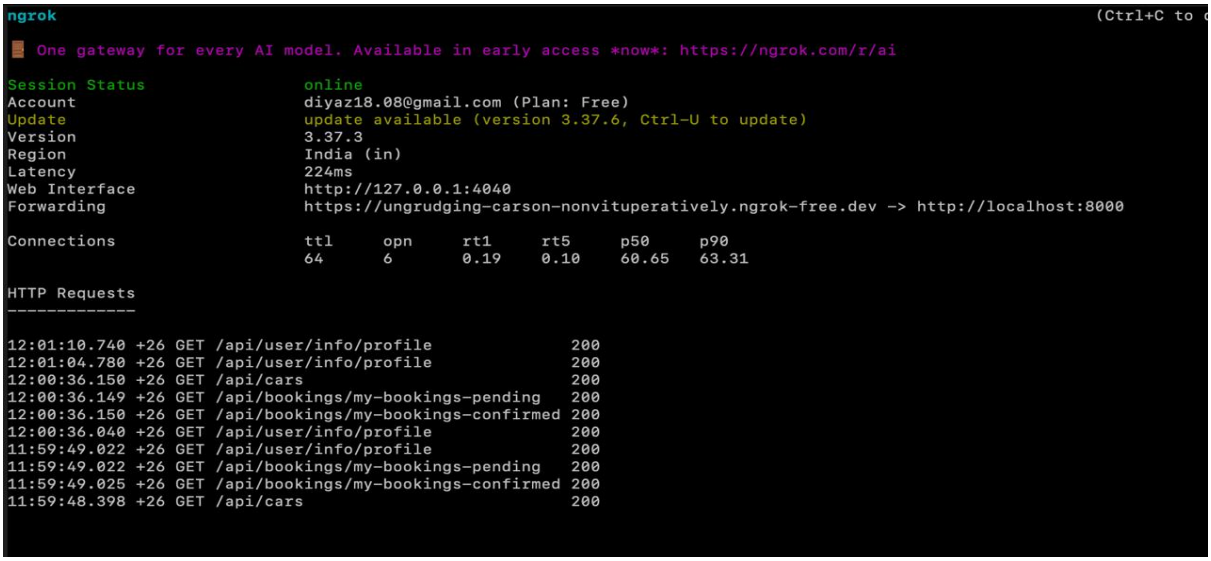


Figure 11. Configuration of the secure Ngrok tunnel for remote mobile-to-backend communication.

6.3 Booking System Testing

The vehicle reservation module represents the core business logic of the Hermes platform. Therefore, rigorous testing was conducted to ensure the reliability and transactional integrity of the booking process. The testing phase focused on evaluating how the backend processes reservation requests under various conditions, particularly focusing on data consistency.

Key test scenarios executed during this phase included:

- **Successful vehicle reservation:** Verifying that a user can successfully book an available vehicle, which correctly triggers the creation of a new relational record in the PostgreSQL database.
- **Concurrency and double-booking prevention:** Simulating simultaneous booking requests from multiple clients for the exact same vehicle. This test ensured the backend correctly handles concurrent transactions, approving the first request and rejecting subsequent attempts to prevent database conflicts.
- **Dynamic state synchronization:** Confirming that immediately after a successful booking, the vehicle's availability status is updated in real-time, automatically removing it from the catalog of available cars for other active users.
- **Data integrity verification:** Ensuring that all booking metadata, including timestamps, user identifiers, and specific vehicle IDs, are accurately and securely recorded in the system.

Testing confirmed the robustness of the system. The implemented database constraints and backend validation logic effectively prevented booking overlaps, guaranteeing a reliable and conflict-free rental experience.

6.4 User Interface Testing

User interface (UI) testing was systematically performed on the mobile application to evaluate its usability, visual consistency, and overall responsiveness. Since the frontend relies on a dynamic architecture, testing also involved assessing the efficiency of screen rendering and state management during active user interactions. The evaluation process concentrated on the following critical aspects:

- **Application navigation and routing:** Testing the fluidity of transitions between major modules (e.g., navigating from the Authentication flow to the Home dashboard, and entering the detailed Vehicle view) to ensure a frictionless user journey.
- **Interactive component functionality:** Verifying that all interactive elements, including action buttons, input fields, and the media picker, respond accurately to touch events and execute the correct underlying functions without delays.

- **Client-side input validation:** Assessing the effectiveness of forms in capturing incorrect, empty, or improperly formatted data before submission, ensuring that users receive immediate and clear visual error messages.

- **Dynamic data rendering:** Confirming that complex data structures received from the API, including vehicle images and specifications, are correctly parsed and aesthetically displayed within scrollable lists and detailed summary cards.
  The comprehensive UI testing concluded that the mobile application delivers a highly responsive and intuitive experience, effectively shielding users from backend complexities while providing clear, logical navigation paths.

7. Discussion

The Hermes project demonstrates how modern software technologies can be combined to create a functional prototype of a car-sharing platform. The successful integration of a mobile application, backend services, and database management allowed the development team to simulate the basic functionality of real-world carsharing systems.

One of the main strengths of the Hermes system is its **modular architecture**. The separation between the mobile frontend and the backend server provides clear structural organization and improves system maintainability. This architectural approach allows different components of the system to be developed and modified independently, which is particularly beneficial for future system expansion and scalability.

Another important aspect of the project is the use of **modern development frameworks and tools**. The flutter framework enables efficient cross-platform mobile development, while REST API communication provides a flexible and scalable way for the mobile application to interact with the backend server. These technologies significantly simplify the development process and allow developers to build complex mobile systems within a relatively short development period.

However, despite the successful development of the prototype, several **limitations of the current system** should be considered.

First, the Hermes application currently operates as a **simplified prototype**, which means that some features expected in a production-level car-sharing system are not yet implemented. For example, the current version of the system does not support real payment processing. In real car-sharing platforms, users must be able to complete secure financial transactions through payment gateways such as credit card systems or digital wallets.

Another limitation of the current system is the absence of **real-time vehicle tracking**. Many commercial car-sharing services use GPS technology to display the exact location of available vehicles on a map, allowing users to locate the nearest car quickly. In the Hermes prototype, pickup locations are predefined rather than dynamically detected.

Additionally, the current system uses a simplified vehicle inventory model. While this approach is sufficient for demonstrating the booking logic and vehicle availability management, a real-world system would require a more advanced inventory management system capable of handling a large number of vehicles and users simultaneously.

Despite these limitations, the Hermes prototype successfully demonstrates the fundamental architecture and functionality required for a car-sharing platform.

Future improvements could significantly enhance the system and bring it closer to real-world applications. Possible improvements include:

a. Integration with **GPS and real-time mapping services** to display nearby vehicles

b. Implementation of **secure online payment systems** for rental transactions

c. Adding **vehicle tracking functionality** for better fleet management

d. Implementing **push notifications** to inform users about booking updates and rental status

e. Expanding the **vehicle inventory system** to support a larger fleet of vehicles

By implementing these improvements, the Hermes system could evolve from a prototype into a more advanced platform capable of supporting real-world car-sharing operations.

8. Conclusion

The Hermes project demonstrates the development of a prototype mobile carsharing application that integrates modern mobile development technologies with backend systems and database management. The primary objective of this project was to design and implement a functional prototype that simulates the basic functionality of a real-world car-sharing service.

Throughout the development process, several key components of the system were successfully implemented. The mobile application provides users with the ability to register, authenticate, browse available vehicles, select pickup locations, and initiate the booking process through an intuitive user interface. The backend system manages the core business logic of the application, including user authentication, vehicle availability management, and booking operations. The PostgreSQL database stores and organizes system data, ensuring that user information, vehicle records and rental transactions can be retrieved and updated efficiently.

One of the main achievements of the Hermes project is the successful integration of multiple technologies into a single working system. The flutter framework allowed the development of a responsive mobile interface, while the java backend provided a reliable platform for processing application logic and managing data communication through REST API endpoints.

The project also demonstrates the importance of proper system architecture when developing modern software applications. By separating the frontend, backend, and database components, the system becomes more scalable and easier to maintain. This architecture allows additional features to be added in the future without requiring significant modifications to the existing system.
In addition to technical implementation, the project provided valuable experience in software development practices such as system analysis, interface design, backend development, database design, and testing. Working on the Hermes system allowed the development team to better understand the process of building a complete software system from initial concept to final prototype.

Despite the successful development of the prototype, several limitations remain. The current system does not include real payment processing, real-time GPS vehicle tracking or advanced vehicle management systems. These features are commonly found in commercial car-sharing platforms and would be necessary for a fully functional production system.

Future improvements to the Hermes application may include:

- integration of real-time GPS tracking for vehicles
- implementation of secure online payment systems
- expansion of the vehicle inventory system
- implementation of push notifications for booking updates
- integration with real map services for location detection

By implementing these improvements, the Hermes prototype could evolve into a more advanced system capable of supporting real-world car-sharing operations.

In conclusion, the Hermes project successfully demonstrates the development of a mobile car-sharing platform prototype. The system integrates mobile application development, backend services and database technologies into a unified architecture that supports the core functionality of vehicle rental systems. The results of this project confirm that modern software development tools provide effective solutions for building scalable and user-friendly mobility applications.

9. References

Turoń, K. (2023) Car-Sharing Systems in Smart Cities: A Review of the Most Important Issues Related to the Functioning of the Systems in Light of Scientific Research. *Smart Cities*, 6(2), pp. 796–808.

Nansubuga, B. (2021) Carsharing: A Systematic Literature Review and Research Agenda. *Journal of Services Marketing*, 32(6), pp. 55–70.

Tao, Z. (2021) Research on Travel Behavior with Car Sharing under Urban Transportation Systems. *Mathematical Problems in Engineering*. Available at: [suspicious link removed] (Accessed: 13 April 2026).

Vătămănescu, E.M. (2024) Connecting Smart Mobility and Car Sharing: A Systematic Literature Review. *Journal of Cleaner Production*. Available at: https://www.sciencedirect.com/journal/journal-of-cleaner-production (Accessed: 13 April 2026).

Flutter (2025) *Flutter Documentation*. Available at: https://docs.flutter.dev (Accessed: 5 March 2026).

Google (2025) *Flutter SDK Overview*. Available at: https://flutter.dev (Accessed: 5 March 2026).

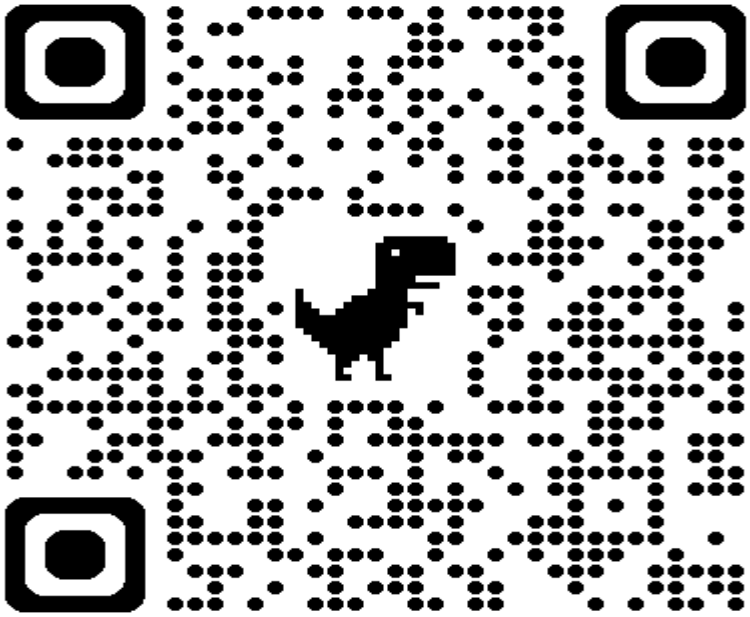
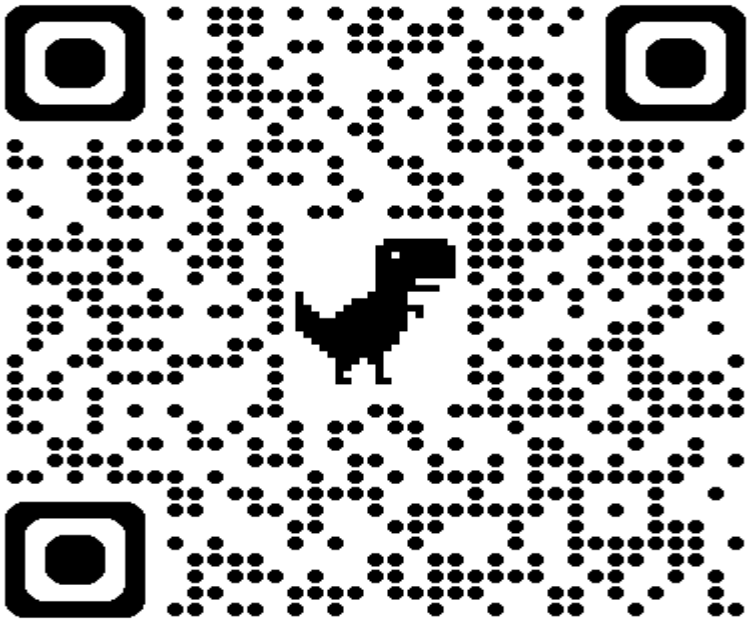
Stošović, S., Stefanović, D., Bogdanović, M. and Vukotić, N. (2022) The Use of the Flutter Framework in the Development Process of Hybrid Mobile Applications. *Knowledge International Journal*, 54(4), pp. 581–587.

Kinari, S.A. and Funabiki, N. (2025) A Guided Self-Study Platform for Flutter Cross-Platform Mobile Programming. *Computers*, 14(2). Available at: https://www.mdpi.com/journal/computers (Accessed: 13 April 2026).

Flutter Team (2025) *Flutter Architectural Overview*. Available at: https://docs.flutter.dev/resources/architectural-overview (Accessed: 5 March 2026).

Wikipedia (2026) *Flutter (software)*. Available at: https://en.wikipedia.org/wiki/Flutter_(software (Accessed: 5 March 2026).

10. Appendices

GitHub	 https://github.com/Demiurg20/Hermes
Video	 https://www.youtube.com/watch?v=ic2DkfUfkQw